My Waves Recommendation System

1. Introduction

The **Wavestube Recommendation API** is a production-ready, high-performance recommendation engine designed to deliver trending, personalized, and content-based video suggestions. Built using **FastAPI**, **FAISS**, and **Sentence-Transformers**, it utilizes a structured CSV-based data pipeline and a Two-Tower retrieval architecture to serve scalable video recommendations.

This document provides a formal technical overview of the API, its features, configuration, setup steps, and operational workflows.

---

2. System Overview

The Wavestube Recommendation API processes structured datasets collected from CSV sources to generate real-time trending feeds, personalized home feeds, and similar-video recommendations. It leverages vector similarity search and deterministic ranking logic to provide reproducible, user-specific results.

---

3. Core Features

3.1 Data-Driven Architecture

The system loads and indexes data from structured CSV files:

- **ugc_videos.csv** – Video metadata
- **ugc_genres.csv** – Genre definitions
- **ugc_genres_video_rel.csv** – Video-to-genre relationships
- datasets:
    - **ugc_watch_history.csv**
    - **ugc_likes.csv**
    - **ugc_watch_later.csv**
    - **ugc_subscriptions.csv**

3.2 Dynamic DataStore

A centralized `DataStore` class:

- Loads, normalizes, and indexes CSV datasets.
- Computes dynamic view counts from watch history.
- Maps genres and channels.
- Supports personalized retrieval and user segmentation.

3.3 Real-Time Trending Feed

Endpoints:

- `/trending` – Global trending videos sorted by total views.
- `/trending/{user_id}` – Optional region filtering + user contextualization.

Features:

- Strict descending view-based ranking.
- Unlimited `limit` parameter support.
- Region-based ranking when creator metadata is available.
  data flow : DataStore → Index → API.

3.4 Personalized Home Feed

`/home/{user_id}` generates a hybrid feed:

- New or inactive users → pure trending.
- Active users → combined trending + personalized results.

Personalization is derived from:

- Recent likes
- Watch history similarity
- Watch-later content
- Subscriptions and related channels

Includes deterministic shuffling and deduplication.

3.5 Similar Video Discovery

`/similar/{video_id}` returns nearest-neighbor similar videos using:

- Vector embeddings
- FAISS index for ANN search

Ensures the queried video is excluded from results.

3.6 Deterministic Shuffling

Consistent hash-based seed (`_deterministic_seed`) ensures:

- User-specific shuffling
- Reproducible paginated results

3.7 Genre Support

Genre metadata strictly matches CSV definitions:

- Each response includes genre IDs and names
- Case-insensitive filtering

3.8 Pagination & Deduplication

- Every feed page is unique and non-overlapping.
- Canonical feed generation ensures stable pagination.

3.9 Scalable Vector Indexing

Integrates with `VideoIndex` for:

- Embedding generation
- Video-to-video similarity
- Hybrid personalized retrieval

3.10 Region & Fallback Handling

When insufficient personalized items exist:

1. Expand trending search (`limit=1000`). Fallback to full catalog as last resort.

Similarity Weighting (for similar video endpoint)

SIMILARITY_WEIGHT_EMBEDDING= 0.4

SIMILARITY_WEIGHT_TITLE= 0.2

SIMILARITY_WEIGHT_TAGS= 0.15

SIMILARITY_WEIGHT_GENRES= 0.15

SIMILARITY_WEIGHT_CATEGORIES=0.1

---

4. Example API Calls

Trending:
curl -X GET "http://localhost:8000/trending?limit=50"

-

Region-Specific Trending:
curl -X GET "http://localhost:8000/trending?region=IN&limit=100"

-

User-Aware Trending:
curl -X GET "http://localhost:8000/trending/{user_id}?limit=50&region=US"

-

Home Feed:
curl -X GET "http://localhost:8000/home/{user_id}?page=1&k=20"

-

Similar Videos:
curl -X GET "http://localhost:8000/similar/{video_id}?k=10"

-

---

5. Environment Configuration (.env)

```
# Reader credentials
host = "your host name"
port = "3306"
user = "your user"
password = "your password"
database = "your database name"

# Data & models
DATA_DIR=wavestube_raw_data
```

```
VIDEO_CSV=ugc_videos.csv
WATCH_HISTORY_CSV=ugc_watch_history.csv
LIKE_CSV=ugc_like_dislike.csv
WATCH_LATER_CSV=ugc_watch_later.csv
SUBSCRIPTIONS_CSV=ugc_subscriptions.csv
CREATORS_CSV=ugc_creators.csv
VIDEO_TAGS_CSV=ugc_video_tags.csv
GENRES_CSV=ugc_genres.csv
CATEGORY_VIDEO_REL_CSV=ugc_category_video_rel.csv

# Embedding / model
MODEL_NAME=sentence-transformers/all-MiniLM-L6-v2
EMBED_DIM=384

# FAISS index
FAISS_INDEX_PATH=./data/video_faiss.index
FAISS_USE_GPU=1

# Misc
TOP_K_DEFAULT=10
DEVICE=cuda
LOG_LEVEL=info
```

---

6. Setup Instructions

Create Virtual Environment
python -m venv venv
source venv/bin/activate

1.

Install Requirements
pip install -r requirements.txt

2.
3. **Edit `.env` file**
    - Set `DATA_DIR`, `MODEL_NAME`, and FAISS GPU settings.

Fetch Data
python3 Database_fetch.py

4.

Build FAISS Index
python3 -m scripts.build_index

5.

Start FastAPI Server
uvicorn app.main:app --host 0.0.0.0 --port 8000

6.

---

7. Production Endpoints

- Similarity:
  - `GET /api/similar/{video_id}?k=10`
- Home Feed:
  - `GET /api/home/{user_id}?k=10&page=1&genre=5`
- Subscription Panels:
  - `GET /api/panels/subscriptions/{user_id}?limit=20`
  - `GET /api/panels/history_similar/{user_id}?k=20`
  - `GET /api/panels/watchlater_similar/{user_id}?k=10`
  - `GET /api/panels/liked_similar/{user_id}?k=20`
- Trending:
  - `GET /api/trending?region=India&limit=20`

---

8. Cron Job Configuration

8.1 Update Data Every 2 Hours

python3 Database_fetch.py

8.2 Rebuild Index (15 Minutes After Data Refresh)

python3 -m scripts.build_index

8.3 Full Cron Script

#!/bin/bash

```
echo "Hello World"
cd /home/ec2-user/wavestube_recsystem/wavestub_recommendation
source venv/bin/activate

echo "Activation Called and completed"

python3 ./database_fetch_sql.py
python3 -m scripts.build_index

echo "restarting the gunicorn"
sudo systemctl restart gunicorn
```

---

9. Conclusion

The Wavestube Recommendation System provides a robust, scalable, and configurable platform for delivering high-quality recommendations. Its combination of deterministic logic, vector similarity, and dynamic data ingestion makes it suitable for production-grade content discovery applications.

TESTING FOR ABOVE IS STILL ONGOING.